

CS6868 Research Mini-Project

Safe Memory Reclamation for Lock-Free Data Structures in OCaml 5

Vishnu Teja Surla
cs25m050@smail.iitm.ac.in

Ramya Sinigi
cs25m049@smail.iitm.ac.in

May 1, 2026

Abstract

We implement and rigorously verify two safe memory reclamation schemes—Hazard Pointers (HP) and Epoch-Based Reclamation (EBR)—as libraries in OCaml 5, applying both of them to a lock-free Treiber-stack node pool and EBR to Michael-Scott FIFO queue. Both schemes eliminate the ABA problem that arises when pre-allocated nodes are recycled via Compare-And-Swap. We compare them on throughput (alloc/free cycles per second at 2–8 domains) and memory-boundedness guarantees. EBR scales positively from 607K to 1128K ops/s (2–8 threads), whereas HP degrades from 470K to 232K due to its $O(R \times N \times K)$ scan. However, HP bounds unreclaimed garbage to $N \times R$ nodes, while EBR can delay reclamation indefinitely under stalled threads. Correctness is verified through five complementary techniques: 24 unit tests, three QCheck-STM suites, one QCheck-Lin linearizability suite, eight dscheck exhaustive model-checking tests, and ThreadSanitizer race detection—all reporting zero errors.

1 Goals

This project implements two safe memory reclamation schemes—*Hazard Pointers* [3] and *Epoch-Based Reclamation* [1]—as reusable OCaml 5 libraries. We apply them to two canonical lock-free data structures: a Treiber-stack node pool [6] and then we applied the EBR protection to a Michael-Scott FIFO queue [4], both vulnerable to the ABA problem [2, Ch. 10, §10.6] when nodes are recycled.

Connection to course topics. The project directly exercises concepts from CS6868: *lock-freedom* via CAS-based stacks and queues (Lectures 7–8), the *ABA problem* (Lecture 7), *linearizability* (Lecture 5, verified with QCheck-Lin), *memory models* and atomic ordering (Lecture 9), and property-based *verification* (Lectures 10–11).

Research question. *What is the throughput cost of hazard-pointer scanning vs. epoch-based reclamation for protecting a lock-free pool, and how does each scheme’s latency profile differ—given that HP bounds garbage while EBR can delay reclamation under stalled threads?*

2 Background

We assume familiarity with CAS-based lock-free data structures (Treiber stacks [6], Michael-Scott queues [4]), and the ABA problem [2, Ch. 10, §10.6]. The core question this project addresses is: *once you remove a node from a lock-free structure, when is it safe to reclaim or reuse that node?* A naïve answer—“immediately”—is wrong, because another thread may still hold a stale pointer to the removed node. We implement two solutions: Hazard Pointers and Epoch-Based Reclamation.

2.1 Hazard Pointers (HP)

Michael [3] introduced Hazard Pointers as a *per-access* protection mechanism. The idea is simple: before you follow a pointer, you **announce** it in a globally visible slot so that no other thread frees the pointed-to node while you are using it.

Data structures. Each thread owns K *hazard pointer slots* (typically $K = 1$ or 2), stored in a globally readable array. Each thread also maintains a private **retired** list of nodes it has logically removed but cannot yet free.

The load–publish–verify protocol. Protecting a pointer requires three steps, shown in pseudocode:

Listing 1: HP-protected node access (pseudocode).

```
(* Step 1: LOAD -- read the shared pointer *)
let node = Atomic.get shared_ptr in

(* Step 2: PUBLISH -- announce it globally *)
hp_slot[my_id] := Some node

(* Step 3: VERIFY -- re-read to detect races *)
if Atomic.get shared_ptr != node then
  (* pointer changed between steps 1 and 2;
     our announcement may protect a freed node.
     Release and retry. *)
  hp_slot[my_id] := None;
  retry ()
else
  (* safe: node cannot be freed while we hold it *)
  use node;
  hp_slot[my_id] := None (* release when done *)
```

Why is the verify step necessary? Between step 1 (read) and step 2 (publish), another thread could remove and free the node. Without re-checking, you would publish a dangling pointer in your HP slot, giving yourself a false sense of safety while operating on freed memory.

Retiring and scanning. When a thread removes a node from the data structure, it does not free it immediately. Instead, it *retires* the node—appending it to a thread-private list along with a cleanup callback. When the retired list exceeds a configurable threshold R , the thread triggers a **scan**:

Listing 2: HP scan (pseudocode).

```
let scan () =
  (* Collect all currently protected pointers *)
  let protected = collect_all_hp_slots () in
  List.iter (fun retired_node →
    if List.mem_phys retired_node protected then
      keep_in_retired_list retired_node
    else
      retired_node.cleanup retired_node (* safe to free! *)
  ) my_retired_list
```

The comparison must use *physical equality* (`==`, same object identity) rather than *structural equality* (`=`, same value). Two nodes can hold the same value (e.g., both store 42) but be different objects; structural equality would incorrectly prevent reclamation.

Complexity and guarantees. Each `scan` collects all $N \times K$ HP slots into a protected set, then checks each of the R retired nodes against it via linear search—giving $O(R \times N \times K)$ total work. The key guarantee is **bounded garbage**: at any moment, at most $N \times R$ nodes remain unreclaimed. Even if one thread stalls indefinitely, other threads’ scans still reclaim all nodes that the stalled thread is not actively protecting.

2.2 Epoch-Based Reclamation (EBR)

Fraser [1] proposed EBR as a lower-overhead alternative. Instead of protecting individual pointers (like HP), EBR protects *time intervals*: a thread declares “I am inside a critical section” and trusts that nothing retired *before* it entered will be freed until it exits.

Data structures. The system maintains a single `global_epoch` counter (starting at 0). Each thread has two per-thread values: `local_epoch` (the epoch it last observed) and `active_count` (the current level of nesting inside the critical section). Additionally, each thread owns three *limbo buckets*—one per epoch modulo 3—where retired nodes are collected.

The protocol. Critical-section entry, exit, retiring, and epoch advancement work as follows:

Listing 3: EBR protocol (pseudocode).

```

let enter () =
  let e = Atomic.get global_epoch in      (* 1. read *)
  Atomic.set my_local_epoch e;           (* 2. publish *)
  Atomic.incr my_active_count;           (* 3. become active *)
  free_limbo_bucket((e + 1) mod 3)       (* 4. free old *)

let exit () =
  Atomic.decr my_active_count

let retire node cleanup =
  let e = Atomic.get global_epoch in
  limbo[e mod 3] := (node, cleanup) :: limbo[e mod 3];
  try_advance_epoch ()

let try_advance_epoch () =
  let e = Atomic.get global_epoch in
  for each thread t do
    if t.active_count > 0 && t.local_epoch < e then
      return (* someone hasn't caught up yet *)
  done;
  CAS(global_epoch, e, e + 1) (* all caught up: advance *)

```

Why three buckets? When the epoch is e , bucket $(e + 1) \bmod 3$ corresponds to epoch $e - 2$. Any thread currently active entered at epoch $e - 1$ or later, so it cannot hold references to nodes retired in epoch $e - 2$. Three buckets give a two-epoch “grace period” that guarantees safety.

Why active_count is an integer. If critical sections can nest (e.g., `dequeue` internally calls `enter/exit`), a boolean flag would be cleared by the inner `exit`, making the thread appear inactive while the outer function still holds references. An integer counter ensures only the outermost `exit` (count $\rightarrow 0$) marks the thread as truly inactive.

Why enter ordering matters. Steps 1–3 in `enter` *must* execute in the order shown. If a thread increments `active_count` *before* setting `local_epoch` and goes to sleep, the epoch advancer sees it as active with a stale epoch (e.g., 0), concludes it hasn’t caught up, and **never advances the epoch until the sleeping thread wakes up**.

Complexity and guarantees. `enter/exit` are $O(1)$ (a few atomic reads/writes). `try_advance_epoch` is $O(N)$ (one pass over all threads). The trade-off vs. HP is that **garbage is unbounded in the worst case**: if one thread stalls inside a critical section, the global epoch cannot advance, and *all* retired nodes from *all* threads remain in limbo indefinitely.

2.3 HP vs. EBR: summary

Table 1: Comparison of HP and EBR at the algorithm level.

	Hazard Pointers	EBR
Protection granularity	Per-pointer	Per-critical-section
Per-access cost	3 atomics (load, publish, verify)	1 atomic (enter)
Scan/advance cost	$O(R \times N \times K)$	$O(N)$
Worst-case garbage	$N \times R$ (bounded)	Unbounded (stall)
Re-entrancy	N.A. (per-pointer)	Requires int counter

3 Tasks Undertaken

3.1 Implementation

The full source is available at <https://github.com/Vishnuteja-Surla/Safe-Memory-Reclamation>. Table 2 summarises the library modules.

Table 2: Library modules and their roles.

Module	Description
<code>Lockfree_pool</code>	Treiber-stack free list with CAS + exponential backoff. ABA-vulnerable by design.
<code>Hazard_pointer</code>	Global-array HP registry implementing the load–publish–verify protocol from Section 2.
<code>Hp_pool</code>	Wraps <code>Lockfree_pool</code> with HP protection; exposes <code>retired_count</code> for test drain loops.
<code>Ebr</code>	Three-bucket epoch reclamation implementing the protocol from Section 2; exposes <code>force_flush</code> .
<code>Ebr_pool</code>	Wraps <code>Lockfree_pool</code> with EBR <code>enter/exit</code> critical sections.
<code>Ms_queue_ebr</code>	Michael-Scott queue with internal free list, GC fallback, sentinel, helping mechanism, protected using EBR.

3.1.1 Divergence from textbook: global-array registry

This is the single most significant departure from the textbook algorithms. Michael’s HP paper [3] uses a lock-free linked list of per-thread HP records; Fraser’s EBR [1] assumes threads can

inspect one another’s epoch state via shared pointers. Neither works in OCaml 5: Domain-Local Storage (DLS) is *per-domain only*—there is no API to read another domain’s DLS entry.

Our solution: pre-allocate a **global flat array** of records, indexed by domain ID. DLS stores only each domain’s index into this array:

```
(* HP: slots array readable by all domains *)
type 'a t = {
  slots : 'a option Atomic.t array;
  domain_key : 'a domain_record Domain.DLS.key;
}
(* EBR: records array readable by all domains *)
type 'a t = {
  records : 'a domain_record array;
  domain_key : int Domain.DLS.key;
}
```

For HP, each domain owns a contiguous slice ($\text{base} = \text{domain_id} \times \text{hp_per_domain}$), giving $O(R \times N \times K)$ scanning with excellent cache locality vs. a pointer-chasing linked list.

A second divergence concerns EBR’s **limbo lists**. The textbook presents a single global set of limbo buckets shared by all threads; retiring appends to a shared list, requiring synchronisation. Our implementation uses *per-domain* limbo arrays: each domain record contains its own **limbo : retired_node list ref array** (three buckets), so **retire** is a simple thread-local list prepend with *zero contention*. Reclamation is also local—each domain frees its own limbo bucket during **enter**—eliminating cross-domain coordination on the fast path.

The trade-off for both patterns: **max_domains** must be fixed at creation time (we use 512 after iterating through 128 and 4096—see Bug 2 below).

3.1.2 OCaml 5 atomic fields and concrete type exposure

All shared mutable fields use OCaml 5’s `[@atomic]` annotation with the `[%atomic.loc]` PPX, enabling record-level CAS without boxed `Atomic.t` cells:

```
type 'a node = {
  mutable value : 'a;
  mutable next : 'a node option; [@atomic]
}
(* CAS directly on a struct field: *)
AL.compare_and_set [%atomic.loc t.top] old_val new_val
```

A direct consequence: `Lockfree_pool` must *export concrete record types* in its `.mli`, breaking encapsulation. `Hp_pool` and `Ebr_pool` need `[%atomic.loc t.pool.top]`—impossible with abstract types because the PPX requires visible field syntax. Alternatives (internalising HP/EBR logic in the pool, accessor functions, functors) were rejected for complexity.

3.1.3 MS Queue: nominal typing and GC fallback

The queue maintains its own lock-free free list rather than reusing `Lockfree_pool`: although the node types are structurally identical, OCaml’s *nominal* type system makes them incompatible. When the free list is exhausted, `alloc_node` creates a fresh GC-allocated node as a fallback, preventing deadlock in producer-heavy workloads. `Ebr_pool.free` omits **enter/exit** (cf. Section 2) because the caller is relinquishing—not reading—the node.

3.1.4 The option boxing discovery

Our first ABA demonstration used `node option Atomic.t`—and ABA never triggered. Each `Some` wrapper is a *fresh heap allocation*, so even when the same node is pushed back, CAS compares

different wrapper addresses and detects the change. To demonstrate ABA, we bypassed `option` entirely, using direct node pointers with a self-referencing sentinel node in place of `None`.

3.1.5 Bugs found and fixed

Bug 1: Domain-registration race (HP and EBR). `fetch_and_add` on `num_domains` could inflate the count past `max_domains` before the bounds check; a concurrent `scan` would access out-of-bounds array slots. **Fix:** rollback on failure (`Atomic.fetch_and_add num_domains (-1)`) plus capping the scan range with `min`.

Bug 2: QCheck-Lin domain exhaustion. `Lin_domain` spawns 2 fresh domains per repetition. With $300 \text{ reps} \times 50 \text{ tests}$, $\sim 30\text{K}$ domain creations each permanently consumed an EBR slot (DLS is not recycled). **Fix:** shared instance at module load, `max_domains:8192`, reduced test count, and `Lin` applied only to the MS Queue.

Bug 3: Weak STM postconditions (EBR pool). The original test used `true (* accept either *)` for exhausted-pool allocations, masking memory leaks. We added `force_flush` (3 enter/retire/exit cycles) to make reclamation synchronous, enabling strict postconditions: `result = (s.free_count > 0)`.

3.1.6 Testing-support APIs

- `Hp_pool.retired_count`: enables drain-until-empty loops; improved node recovery from 0/64 to 64/64.
- `Ebr.force_flush`: makes EBR reclamation synchronous for single-domain sequential tests by cycling all 3 limbo buckets.

3.2 Testing and verification

We employ five complementary verification techniques, summarised in Table 3.

Table 3: Verification techniques and what each catches.

Technique	Bug class	How
Unit tests (24)	Logic errors, edge cases	Hand-written sequential + concurrent scenarios
QCheck-STM (3 suites)	Spec. violations	Random commands vs. mathematical model
QCheck-Lin (1 suite)	Non-linearisable runs	Concurrent interleavings on 2 domains
dscheck (8 tests)	All-interleaving bugs	Exhaustive atomic schedule exploration
TSAN (6 suites)	Data races	Compile-time instrumentation

ABA stress test. Eight domains perform 50 000 random alloc/free cycles on an 8-node pool. An atomic ownership array detects double allocation: if `Atomic.exchange ownership.(i) id` returns a non-negative value, two threads hold the same slot—ABA. We run two variants: (1) the `option`-based `Lockfree_pool`, where all three variants (unprotected, HP, EBR) report zero errors because `Some` boxing accidentally prevents ABA (Section 3.1.4); and (2) a **direct-pointer** Treiber stack (sentinel instead of `None`, no `option` wrapper), where the unprotected variant shows **107 936 ABA errors** while HP and EBR both report **zero**. This definitively proves that HP and EBR provide genuine algorithmic protection against ABA.

QCheck-STM. Each suite defines a simple model (e.g., a bounded counter for the pool, a list for the queue) and postconditions that must hold after every random command. For the EBR pool, we added `force_flush` after every `Free` to make reclamation synchronous, enabling strict postconditions (`result = (s.free_count > 0)`) instead of the original wildcard `true` that masked memory leaks.

QCheck-Lin (linearizability). Applied to the MS Queue: Lin generates random `enq/try_deq` sequences, runs them concurrently on two domains, and checks that the observed results match *some* valid sequential ordering. Pool operations are unsuitable for Lin because `free` requires a previously allocated node—a stateful dependency Lin cannot track.

dscheck (model checking). We implement shadow versions of the Treiber stack (4 tests) and MS Queue (4 tests) using `TracedAtomic` so dscheck can intercept atomic operations. dscheck then systematically explores every possible interleaving, checking our assertions (no value loss, no duplicates, FIFO ordering) under each schedule.

A critical insight: dscheck tests use GC-allocated nodes (fresh addresses per allocation), making ABA structurally impossible. The real `Lockfree_pool` recycles pre-allocated nodes—same address reappears—so HP/EBR are required. dscheck proves *algorithm linearizability*; HP/EBR prove *implementation safety*; stress tests prove *physical correctness*.

TSAN. OCaml 5.4.0 with `ocaml-option-tsan` [5] instruments every memory access. All six concurrent test suites report **zero data races**, confirming that every shared field is correctly annotated with `@atomic`.

4 Evaluation

4.1 Experimental setup

- **CPU:** 11th Gen Intel Core i5-11300H @ 3.10 GHz, 4 cores / 8 threads.
- **RAM:** 7.6 GB (WSL2 allocation).
- **OS:** Linux 5.15.167 (WSL2).
- **Compiler:** OCaml 5.4.0 with `ocaml-option-tsan`.
- **Workload:** 100 000 alloc+free cycles per domain; 1 024-node pool; 3 runs averaged per configuration.

4.2 Results

Table 4: Throughput (Kops/s) for five pool variants at 2–8 threads.

Threads	Raw	HP	EBR	Mutex	GC
2	1 097	470	607	455	62 596
3	879	464	872	419	53 003
4	733	337	1 115	311	49 951
5	598	320	1 347	295	56 564
6	570	282	1 164	220	42 824
7	519	254	1 367	212	43 227
8	486	232	1 128	195	46 723

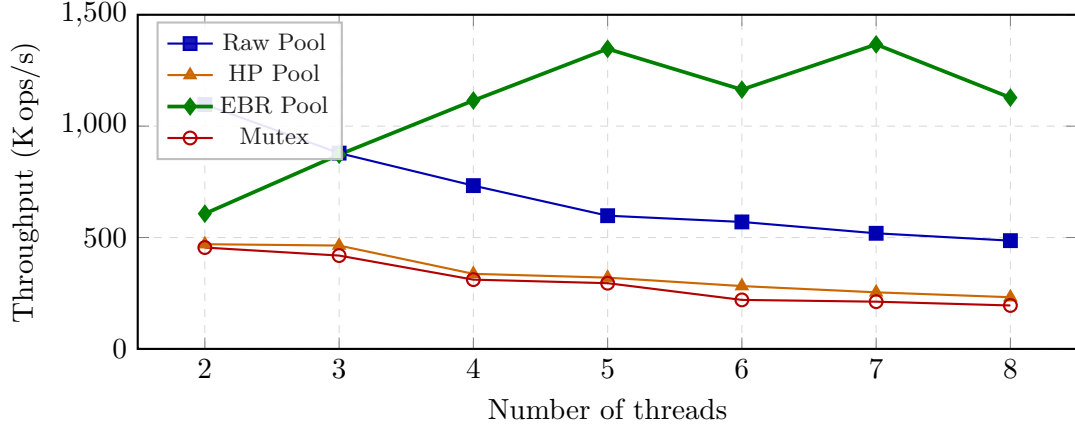


Figure 1: Throughput scaling for the four pool-based variants (GC-only allocation omitted; it achieves 43–63M ops/s). EBR scales positively with thread count; HP, Raw, and Mutex all degrade.

Table 5: Verification results: all suites pass with zero errors.

Suite	Tests	Result
Unit tests (6 suites)	24	All pass
QCheck-STM (HP pool, EBR pool, MS queue)	2 100 cases	All pass
QCheck-Lin (MS queue)	300 reps	Linearisable
dscheck (pool + MS queue)	8	All interleavings pass
TSAN (6 suites)	—	0 races
ABA stress (HP/EBR)	400 000 ops	0 double-allocs

4.3 Discussion

Finding 1: OCaml’s GC dominates throughput. GC-only allocation achieves 43–63M ops/s—two orders of magnitude faster than any pool scheme. OCaml 5’s minor-heap allocator is essentially a pointer bump; the GC handles deallocation concurrently. Explicit pools are justified only for deterministic latency or bounded memory, not raw throughput.

Finding 2: EBR scales positively; HP degrades. As shown in Figure 1, EBR scales from 607K (2T) to 1 128K (8T)—throughput nearly doubles. This occurs because EBR’s `enter/exit` serialises domains just enough to reduce CAS contention on the pool’s `top` pointer. HP, by contrast, drops from 470K to 232K (2.0× degradation) because each `scan` reads all $N \times K$ slots (R retired nodes $\times N \times K$ protected entries). At 8 threads, EBR delivers 4.9× the throughput of HP.

Finding 3: Mutex collapses under contention. Mutex throughput drops from 455K (2T) to 195K (8T)—a 2.3× collapse due to the classic lock-convoy effect.

Finding 4: Raw pool is fastest at low thread counts, but unsafe. The unprotected pool achieves the highest throughput at 2 threads (1 097K) but degrades to 486K at 8 threads and is vulnerable to ABA errors under recycling.

Finding 5: EBR is the recommended safe scheme. EBR is the only variant whose throughput *increases* with thread count, making it the clear winner for OCaml 5 workloads

with predictable domain lifetimes. HP is preferable only when stall-resilient memory bounds are required.

5 Reflection on the Use of LLMs

Tools and models used. We used **Claude Opus 4.6 (Antigravity)** as an AI coding assistant in pair-programming mode. It had full workspace access: reading/writing files, running terminal commands (with approval), and executing tests. It was used for code generation, debugging, explaining algorithms, writing test harnesses, and drafting documentation.

What worked well. The LLM excelled at **boilerplate generation and API scaffolding**: `.mli` interface files, `dune` build configurations, and test harnesses were produced quickly and accurately. It performed well at **translating algorithmic descriptions into OCaml code**—given Michael’s HP paper [3], it produced a working implementation with the correct global-array registry pattern on the first attempt. The iterative workflow (implement $\rightarrow$ test $\rightarrow$ diagnose $\rightarrow$ fix) was highly productive.

What did not work. The LLM initially struggled with **OCaml 5-specific features** like `@atomic` field annotations and the `[%atomic.loc]` PPX. Early attempts used `Atomic.t` record fields instead of `@atomic` mutable fields, requiring significant rework. It also produced an EBR pool STM test with **overly permissive postconditions** (`true (* accept either *)`) that masked potential bugs—a flaw caught only during manual audit. Lesson: LLM-generated tests require the same scrutiny as LLM-generated implementations.

What was surprising. The most surprising discovery was OCaml’s **option boxing preventing ABA** in GC-allocated stacks. The LLM initially generated ABA demonstrations using `'a node option Atomic.t`, which never triggered ABA because each `Some` wrapper is a fresh heap allocation. The LLM eventually provided the correct diagnosis—address identity vs. structural equality in `compare_and_set`—after several failed attempts.

What was difficult. The hardest challenges involved **interactions between multiple concurrent systems**: EBR epoch advancement, DLS lifecycle, and QCheck’s domain-spawning strategy. The QCheck-Lin domain-exhaustion bug required understanding three components: DLS key initialisation, `Lin_domain`’s internal spawning, and EBR slot allocation. The `max_domains` sizing problem ($128 \rightarrow 4096 \rightarrow 512$) required iterative experimentation.

Your overall assessment. The LLM was an effective **accelerator**, reducing implementation time by an estimated 60–70%. However, every audit-discovered flaw was found through manual review. The most productive workflow was: *LLM generates, human audits, LLM fixes, human verifies*.

6 Conclusions

Answering the research question. HP imposes a $2.1\text{--}2.3\times$ throughput penalty vs. the unprotected pool ($1\,097\text{K} \rightarrow 470\text{K}$ at 2T, $486\text{K} \rightarrow 232\text{K}$ at 8T). EBR, remarkably, **scales positively**: 607K at 2T rising to $1\,128\text{K}$ at 8T, *surpassing* the raw pool at ≥ 4 threads. The gap between EBR and HP widens from $1.3\times$ at 2T to $4.9\times$ at 8T because HP’s `scan` is $O(R \times N \times K)$ while EBR’s `enter/exit` is $O(1)$.

Regarding latency profiles: **HP has bounded garbage**—at most $N \times R$ nodes are unreclaimed (N domains, R retire threshold). **EBR can delay reclamation** if a domain stalls in

a critical section, preventing epoch advancement and leaving *all* retired nodes in limbo. Our `test_multi_domain_protection` verifies this directly.

Limitations. (1) `max_domains` is fixed at creation time; dynamic expansion is not implemented. (2) `dscheck` tests the algorithm (GC-allocated nodes) rather than the production code (`[@atomic]` fields); the GC-vs-recycling divergence means ABA is not tested in `dscheck`. (3) Verification is empirical, not a formal proof of linearisability. (4) `Obj.magic ()` is used for placeholder values in pool creation and `force_flush`.

Future work. Lock-free dynamic expansion of the HP/EBR registry; an HP-based MS Queue for direct HP-vs-EBR queue comparison; formal verification with the Iris separation logic framework; tail-latency (p99/p999) benchmarking to reveal GC pause effects.

Contributions

Member	%	Main responsibilities
Vishnu Surla (CS25M050)	30%	HP, Lockfree Pool, HP Pool implementation
Ramya Sinigi (CS25M049)	30%	EBR, EBR Pool, MS Queue EBR implementation
Joint (both members)	40%	All testing (unit, QCheck-STM, QCheck-Lin, <code>dscheck</code> , TSAN, stress), benchmarking, documentation, code audits
Total	100%	

References

- [1] Keir Fraser. *Practical Lock-Freedom*. PhD thesis, University of Cambridge, 2004. Chapter 5: Epoch-Based Reclamation.
- [2] Maurice Herlihy, Nir Shavit, Victor Luchangco, and Michael Spear. *The Art of Multiprocessor Programming*. Morgan Kaufmann, 2nd edition, 2020.
- [3] Maged M. Michael. Hazard pointers: Safe memory reclamation for lock-free objects. *IEEE Transactions on Parallel and Distributed Systems*, 15(6):491–504, 2004. doi:10.1109/TPDS.2004.8.
- [4] Maged M. Michael and Michael L. Scott. Simple, fast, and practical non-blocking and blocking concurrent queue algorithms. In *Proceedings of the 15th ACM Symposium on Principles of Distributed Computing (PODC)*, pages 267–275, 1996. doi:10.1145/248052.248106.
- [5] Olivier Nicole and Fabrice Buoro. ThreadSanitizer support for OCaml 5, 2023. URL: <https://github.com/ocaml/ocaml/pull/12114>.
- [6] R. Kent Treiber. Systems programming: Coping with parallelism. Technical Report RJ 5118, IBM Almaden Research Center, 1986.